

GLOBALRAIN

Practices for Secure Software Report

Table of Contents

DOCUMENT REVISION HISTORY 3
CLIENT 3
INSTRUCTIONS ..... ERROR! BOOKMARK NOT DEFINED.
DEVELOPER 3
1. ALGORITHM CIPHER 3
2. CERTIFICATE GENERATION 4
3. DEPLOY CIPHER 5
4. SECURE COMMUNICATIONS 6
5. SECONDARY TESTING 7
6. FUNCTIONAL TESTING 9
7. SUMMARY 11
8. INDUSTRY STANDARD BEST PRACTICES 11

Document Revision History

Version	Date	Author	Comments
1.0	October 17, 2026	Aaron Sceviour	Initial version

Client



Developer
Aaron Sceviour

1. Algorithm Cipher

For Artemis Financial, I chose two cryptographic tools that work together. SHA-256 is the hash function that creates the checksum used to verify data. RSA with a 2048-bit key pair is the algorithm behind the certificate that secures the web connection over HTTPS. They do different jobs. A hash is one way. It cannot be reversed, and it shows whether data has changed. An encryption cipher scrambles data so only someone with the right key can read it. Artemis handles financial information, so it needs data that stays private and arrives unchanged.

An encryption algorithm cipher protects confidentiality. It takes readable data and a key and produces unreadable data. A good cipher is public and well tested, and its strength comes from the key, not from keeping the method secret. The goal for Artemis is that someone who intercepts traffic between a client and the server cannot read it or quietly alter it.

Hash functions and bit levels. SHA-256 always outputs 256 bits, no matter how large the input is. Changing one character of the input changes the whole output. The data string in my project is "Aaron Sceviour: Hello World Check Sum!" and its checksum is ec48b4a150f90cf9aa09834a808861947d2f71205bdeefbaccc6bc1f2e0bcaa5. A longer output makes it much harder for an attacker to find two different inputs with the same hash, which is called a collision. MD5 produces 128 bits and SHA-1 produces 160 bits. Researchers have produced real collisions for both, so they should not be used to prove data is unchanged. SHA-256 is part of the SHA-2 family, and there is no known practical collision attack against it.

Random numbers. Cryptographic keys come from random values. If an attacker can guess those values, they can rebuild the key. When keytool created my RSA key pair, it used Java's secure random number generator, which was built for this purpose. HTTPS also uses fresh random values in every handshake, so each session gets its own keys.

Symmetric and asymmetric algorithms. A symmetric cipher such as AES uses one shared key to encrypt and decrypt. It is fast, but both sides need a safe way to share the key. An asymmetric algorithm such as RSA uses a public key and a private key. The public key can go to anyone, and the private key stays on

the server. It is slower. HTTPS uses both. The server's RSA key pair proves its identity and helps both sides agree on session keys. A symmetric cipher then encrypts the actual traffic.

Digital signatures and certificates. A digital signature combines a hash with an asymmetric key. The certificate I generated is signed with SHA384withRSA, which means the certificate contents are hashed, and the hash is signed with the RSA private key. My certificate is self-signed, so no outside authority vouches for it. That is fine for testing. In production, Artemis would need a certificate signed by a trusted certificate authority.

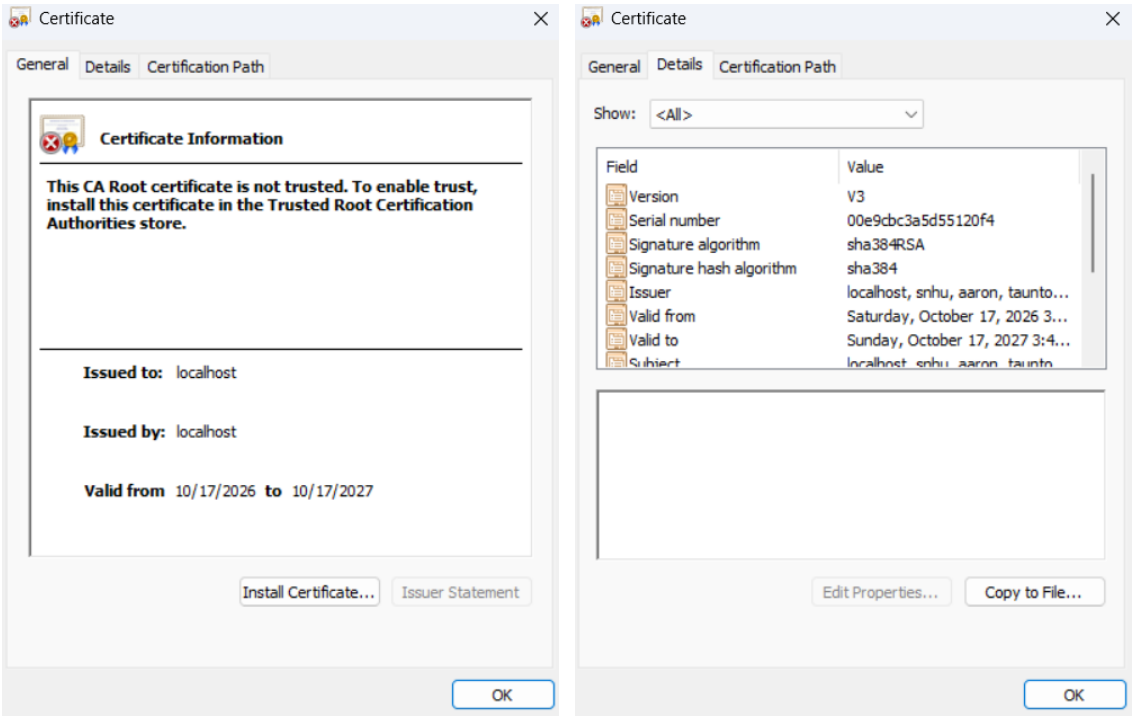
History and current state. Encryption standards change as computers get faster and attacks improve. DES used a 56-bit key and became too easy to break, so AES replaced it. MD5 and SHA-1 were retired for security use after collisions were found. RSA keys of 1024 bits are no longer considered safe, and 2048 bits is the common minimum today. Because of this history, I recommend that Artemis review its algorithms on a schedule and be ready to move to longer keys or elliptic curve options as guidance changes.

2. Certificate Generation

I created a keystore with Java's keytool. The first command generated an RSA key pair with a 2048-bit key, stored it in a PKCS12 keystore named keystore.p12, and made the certificate valid for 365 days. I used the alias artemis and entered localhost as the common name, since that is the host name the browser checks. Keytool asked for the keystore password at a prompt, so it does not appear in the command.

```
keytool -genkeypair -alias artemis -keyalg RSA -keysize 2048 -storetype PKCS12 -keystore keystore.p12 -validity 365
keytool -exportcert -alias artemis -file serverCertificate.cer -keystore keystore.p12
```

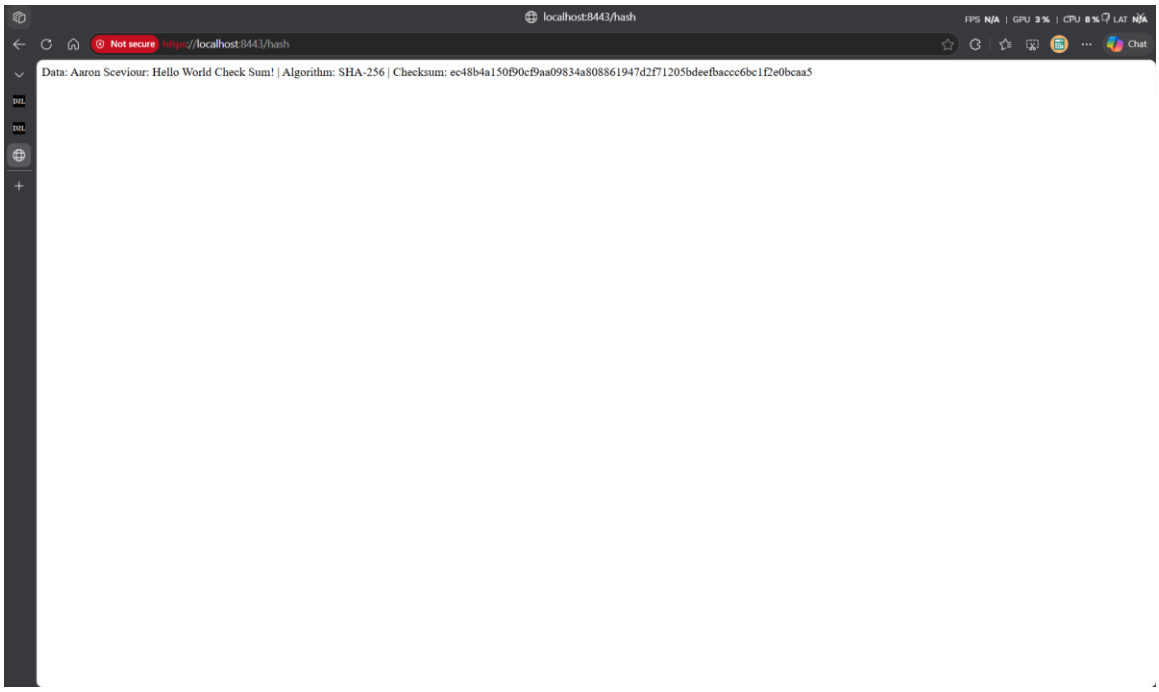
The second command exported the public certificate to serverCertificate.cer. Only the public certificate is in that file. The private key stays in the keystore. The screenshots below show the CER file. The General tab says the certificate is not trusted, because I signed it myself. The Details tab shows the signature algorithm sha384RSA, the validity dates from October 17, 2026 to October 17, 2027, and my information in the issuer field.



3. Deploy Cipher

I added a new class, ServerController, to the project. It is a rest controller with a GET route at /hash. When the route is called, the code hashes the string "Aaron Sceviour: Hello World Check Sum!" with SHA-256 and returns the data, the algorithm name, and the checksum as text. Java's MessageDigest class does the hashing. The raw bytes are converted to a hexadecimal string, with two characters per byte so leading zeros are kept. I chose SHA-256 over MD5 and SHA-1 for the reasons in the Algorithm Cipher section. The checksum shown in the browser matches the value an independent SHA-256 tool produces for the same string.

A checksum proves the data did not change. It does not prove who sent it. An attacker who can change the data could also change the checksum. To close that gap, Artemis should deliver the checksum over HTTPS, as this project does, and use a keyed hash (HMAC) or a digital signature if the sender's identity matters.

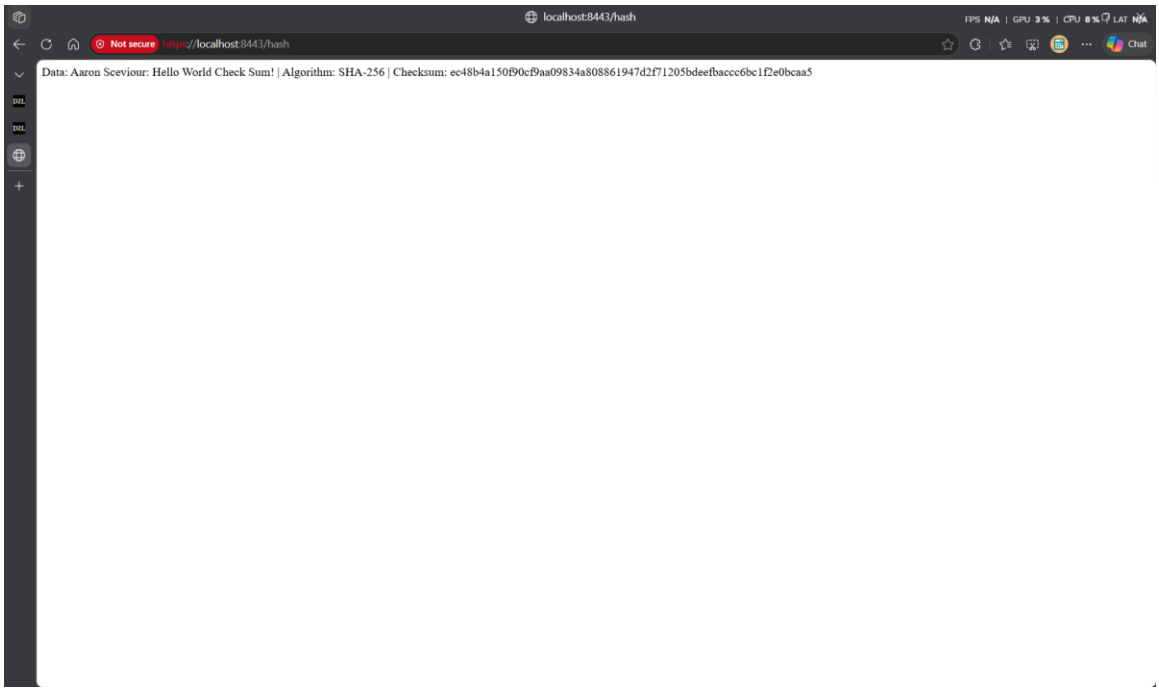


4. Secure Communications

I turned on HTTPS by adding the keystore settings to application.properties. The server runs on port 8443. These are the settings, with the password hidden:

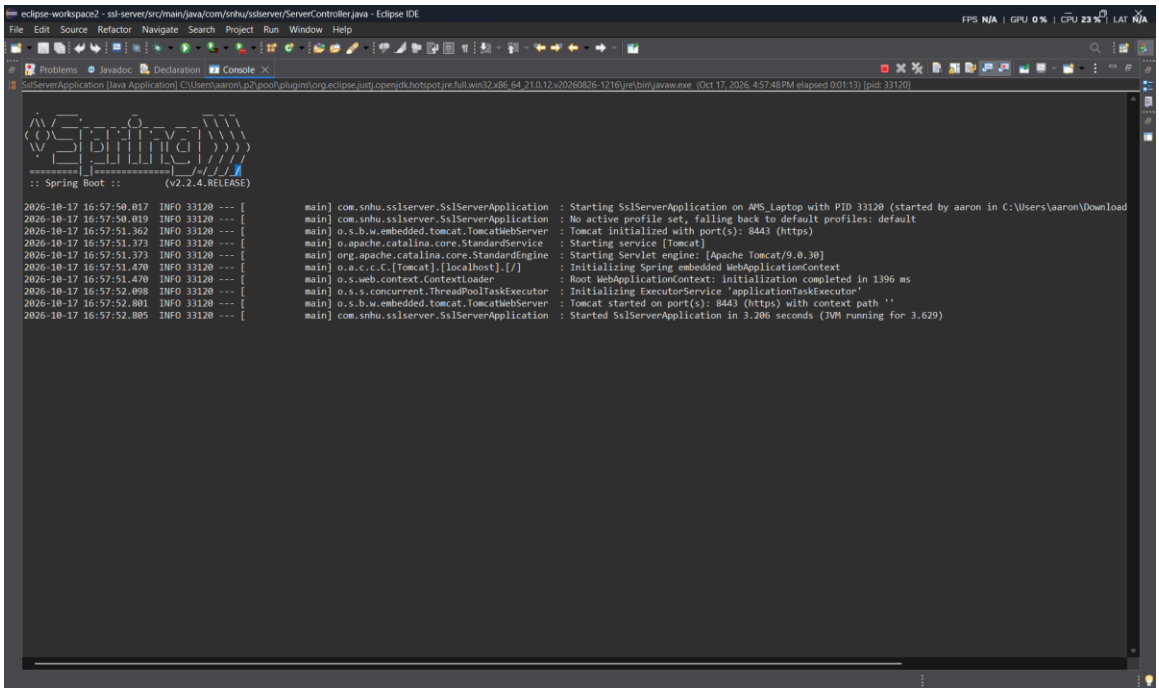
```
server.port=8443
server.ssl.key-alias=artemis
server.ssl.key-store-password=********
server.ssl.key-store=classpath:keystore.p12
server.ssl.key-store-type=PKCS12
```

The alias and keystore type match the keytool command. The classpath prefix tells Spring to look for the keystore in src/main/resources. The screenshot shows the /hash page loading at https://localhost:8443/hash. Edge labels the page "Not secure" because the certificate is self-signed and the browser has no trusted authority to confirm it. The connection is still encrypted with TLS. A certificate signed by a trusted authority would remove the warning.



5. Secondary Testing

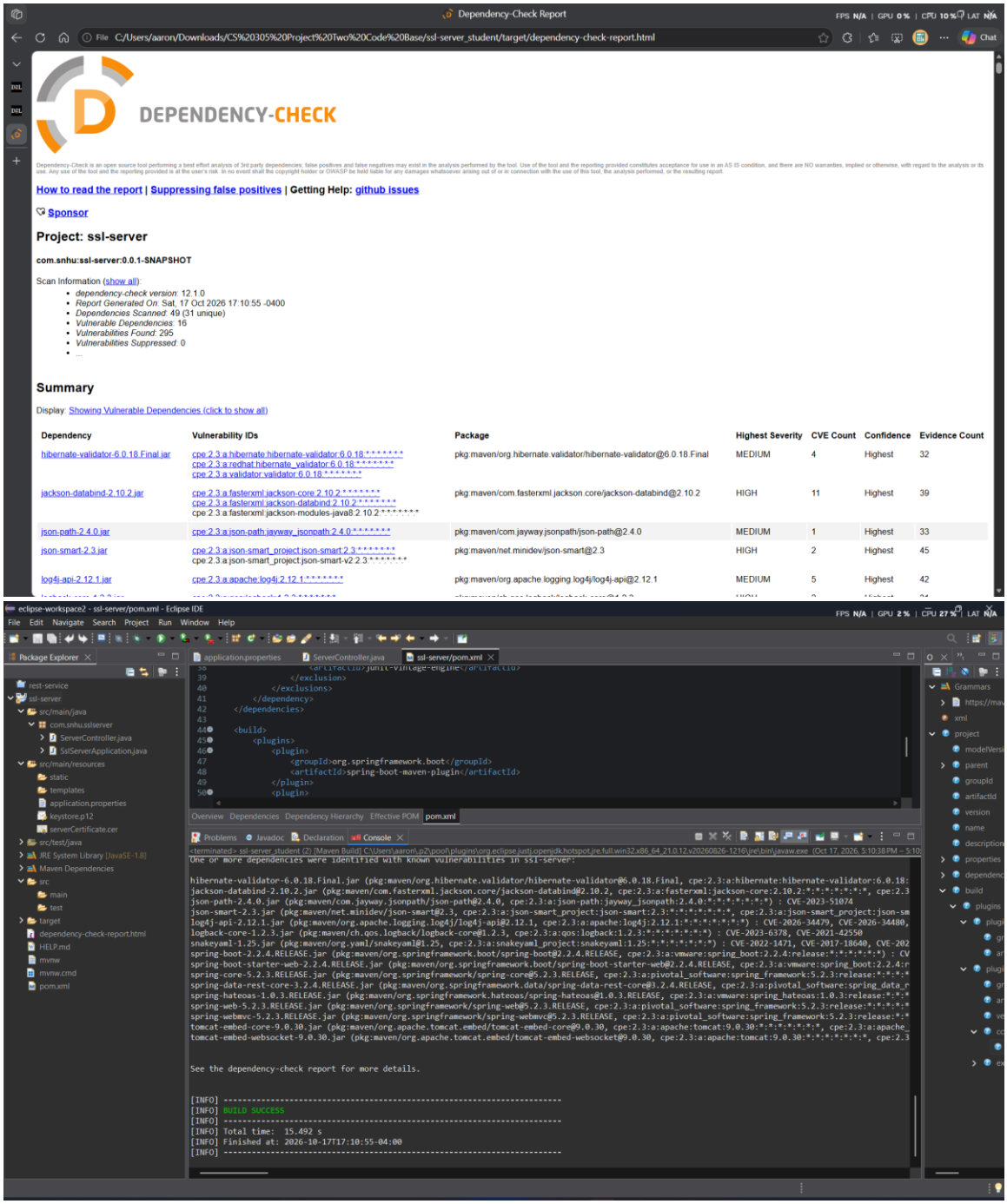
I ran the refactored application, and it started without errors. Tomcat started on port 8443 with HTTPS. Then I ran OWASP Dependency-Check through Maven with the verify goal.



The first scan run failed. The dependency-check plugin in the starter project was version 5.3.0, and it tried to download vulnerability data from feeds that the National Vulnerability Database no longer serves, so it returned a 403 error. I updated the plugin to version 12.1.0, the same version I used in Module Six. A second run failed because the Sonatype OSS Index analyzer needs an account and returned a 401 error. I

turned that one analyzer off in the plugin configuration. The main scan against the National Vulnerability Database ran fine, and the build finished with BUILD SUCCESS.

The scan checked 49 dependencies (31 unique). It found 16 vulnerable dependencies and 295 vulnerabilities, and I suppressed none. Examples include jackson-databind 2.10.2 (High, 11 CVEs), json-smart 2.3 (High, 2 CVEs), hibernate-validator 6.0.18.Final (Medium, 4 CVEs), and log4j-api 2.12.1 (Medium, 5 CVEs). The list also includes snakeyaml, logback-core, several Spring Framework modules, and Tomcat 9.0.30.

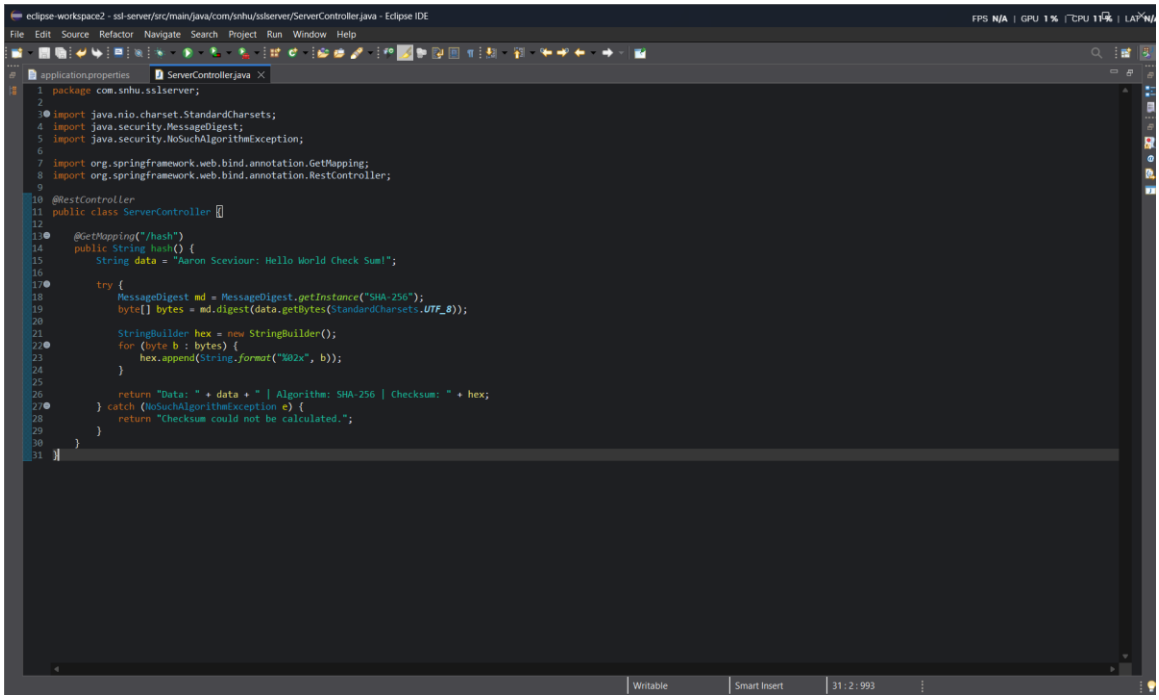


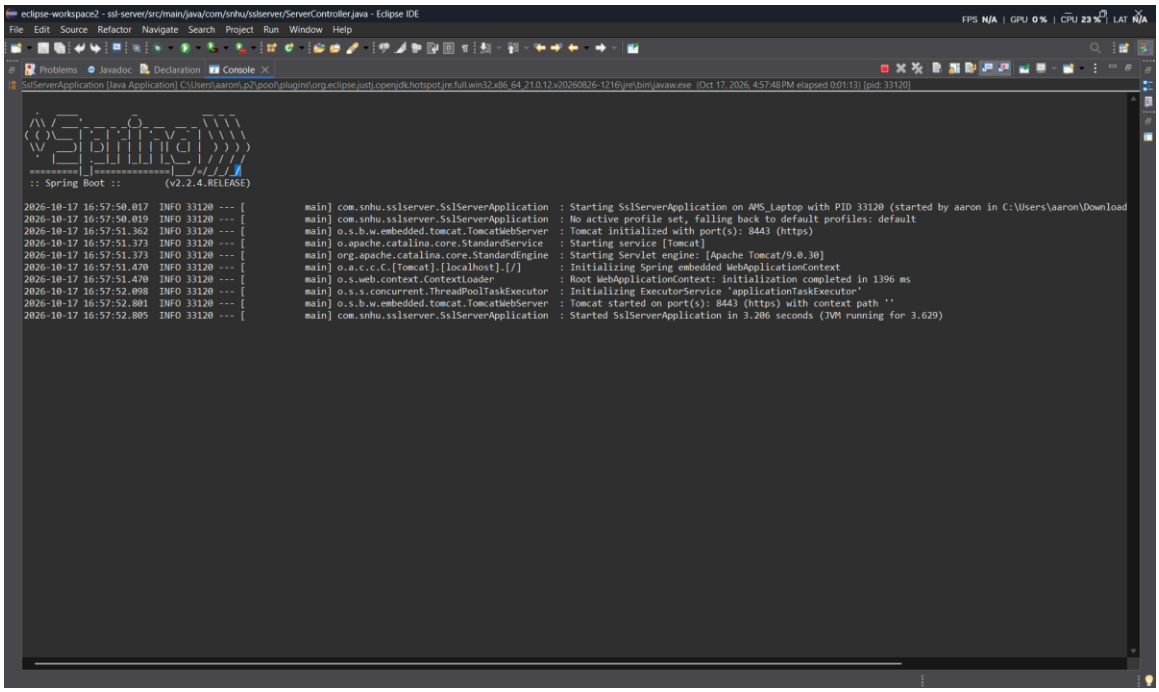
All these libraries came with the starter project through Spring Boot 2.2.4, which is from 2020. I added no dependencies. ServerController imports only classes from the Java standard library and Spring web annotations that were already in the project. Because of that, my refactoring did not introduce any new vulnerable libraries. The plugin version change affects only the build tool, not the application.

The fix for these findings is to upgrade Spring Boot to a current supported version, which brings newer versions of Spring, Tomcat, Jackson, and the others. Then I would rerun the scan and the tests. I did not do that here because a major version upgrade can break working code and is outside the scope of this assignment. Some findings in these reports can also be false positives, so each one should be reviewed before it is fixed or suppressed.

6. Functional Testing

I reviewed the refactored code by hand for syntax, logic, and security problems. The screenshots show the code and the application running without errors.





Syntax. The project compiles, and the Maven build ran the project's test, which passed with no failures.

Logic. The hex conversion uses two characters per byte, so leading zeros are kept. The code names the UTF-8 charset when it turns the string into bytes, so the checksum is the same on every machine. One problem came up during testing. A second run of the application failed with "Port 8443 was already in use" because the first run was still going. I stopped the first run and started again, and it worked.

Security. The /hash route takes no input from the user. The string is fixed in the code, so there is nothing to inject into. The catch block returns a generic message and no stack trace, which avoids leaking internal

details. It should also log the error onto the server, since a silent failure would go unnoticed. The keystore password is stored in plain text in application.properties, and the data string is hard coded. I will cover those fixes in the Industry Standard Best Practices section.

7. Summary

Artemis Financial needed a secure way to verify data and a secure connection for its web application. I refactored the starter project to add both. I created an RSA 2048-bit keystore and self-signed certificate with keytool, turned on HTTPS on port 8443 through application.properties, and added a /hash route that returns a SHA-256 checksum. I then ran the application, reviewed the code by hand, and scanned the dependencies with OWASP Dependency-Check.

Here is how I handled each area from the vulnerability assessment process flow diagram.

Cryptography: Changed. I used SHA-256 for the checksum and an RSA 2048-bit key pair for the certificate, instead of weaker options like MD5 and SHA-1.

Client and server: Changed. The server now uses HTTPS with a keystore, so traffic between the browser and the server is encrypted.

Input validation: Reviewed. The new route takes no user input, so nothing needed validating. If Artemis lets users submit data to hash later, that input must be validated first.

APIs: Reviewed. The only route I added is one read-only GET request served over HTTPS.

Code error: Reviewed. The error handling returns a generic message and leaks nothing. I recommend adding server-side logging.

Code quality: Reviewed, with recommendations. The hashing logic could move into its own service class, and the data string could become a constant or a setting.

Encapsulation: Reviewed, with recommendations. The keystore password is in plain text and should be moved out of the code.

I used Eclipse, Maven, keytool, Spring Boot, and OWASP Dependency-Check. I added no new dependencies. The scan flagged 16 libraries that came with the starter project, and the main follow up is to upgrade Spring Boot, which I did not do in this assignment.

8. Industry Standard Best Practices

These are the practices I would use to keep Artemis Financial's software secure, based on what I found in this project.

1. Keep secrets out of the code. My keystore password is in plain text inside the project. That is fine for coursework, but not for production. If the project reached a public repository, anyone could read the password and open the keystore. A real company would use environment variables or a secrets manager, and keep keystore files out of version control. In Spring, the property can point to a variable such as ${KEYSTORE_PASSWORD}, which the server supplies at startup.
2. Use trust certificates and strong protocols. Production needs a certificate signed by a trusted authority, plain HTTP traffic redirected to HTTPS, and the private key stored securely. Algorithms should be reviewed on a schedule, because standards change, as the history of DES, MD5, and SHA-1 shows.

3. Keep dependencies updated and scan them often. The scan flagged 295 vulnerabilities in 16 libraries from a 2020 starter project. Artemis should upgrade on a schedule, run a scan like Dependency-Check in the build pipeline, and review each finding before fixing or suppressing it. The scan tool needs maintenance too. My first run failed because the old plugin version could no longer download vulnerability data.
4. Validate input and handle errors safely. Any data that comes from a user should be checked for length, type, and allowed characters before the code uses it. Errors should show users a generic message and write the details to a server log.
5. Use the right tool for each security goal. A checksum shows that data did not change, but not who sent it. Where identity matters, use a keyed hash or a digital signature, and deliver the checksum over a secure channel.
6. Review and test regularly. Manual review, automated scans, and functional testing catch different kinds of problems, and I used all three here. On a team, a second person should review each change before it is released.